

Universidad Autonoma de Yucatan  
Facultad de Matematicas  
Ingenieria de Software  
Teoria de Lenguajes de Programacion

# Motor de Inferencia Logica como Servicio

Paradigmas integrados:

Logica (Prolog)

Funcional (JS)

Asincrona (Node.js)

## INTEGRANTES

Cabrera Dzul Rolando Emmanuel  
Castañeda Euan Alfredo Alexander  
Parra Canche Paola Lizeth

## PROFESOR

Alejandro Pasos Ruiz

2026

## 1 Introduccion

Este proyecto implementa un **Motor de Inferencia Logica como Servicio**, un sistema que expone una API REST capaz de ejecutar consultas sobre una base de conocimiento escrita en Prolog. El cliente envia una consulta en sintaxis Prolog y recibe como respuesta los hechos inferidos automaticamente por el motor.

El sistema demuestra la integracion practica de tres paradigmas fundamentales de la programacion moderna: logico, funcional y asincrono. Cada paradigma aporta una capa diferente al sistema, logrando una arquitectura limpia, modular y expresiva.

### Objetivo central

Construir un servicio web donde la logica de negocio (contratos, penalizaciones, clientes) se expresa como reglas declarativas en Prolog, y el servidor Node.js actua como intermediario asincrono entre el cliente HTTP y el motor de inferencia.

## 2 Paradigmas de Programacion

| Paradigma | Donde se aplica                                            | Caracteristica clave                                   |
|-----------|------------------------------------------------------------|--------------------------------------------------------|
| Logico    | knowledge.pl, Tau Prolog                                   | Declara QUE, no COMO. Inferencia automatica.           |
| Funcional | normalizeQuery(),<br>extractBindings(),<br>formatSuccess() | Funciones puras sin efectos secundarios. Composicion.  |
| Asincrono | loadKnowledgeBase(),<br>runPrologQuery(), endpoints        | async/await y Promises para no bloquear el event loop. |

### 2.1 Programacion Logica (Prolog)

En la programacion logica, el programador declara *que* es verdadero, no *como* computarlo. El motor Prolog se encarga de la inferencia mediante unificacion y busqueda por retroceso (backtracking).

Ejemplo de regla declarativa en el proyecto:

```
penalty_applicable(Contract) :-  
    contract(Contract),  
    contract_days_overdue(Contract, Days),  
    Days > 30.
```

Esta regla se lee: "Se aplica penalizacion a un Contrato si ese Contrato existe y lleva mas de 30 dias vencido." El motor infiere automaticamente a que contratos aplica sin necesidad de escribir bucles o condicionales.

## 2.2 Programacion Funcional (JavaScript)

La capa de procesamiento de entrada y salida usa funciones puras: sin efectos secundarios, sin modificacion de estado externo, y con resultado determinista para los mismos argumentos.

```
const normalizeQuery = (raw) => {  
    if (typeof raw !== "string") return null;  
    const trimmed = raw.trim();  
    if (!trimmed) return null;  
    return trimmed.endsWith(".") ? trimmed : `${trimmed}.`;  
};
```

Otras funciones puras: **formatSuccess()** y **formatError()** construyen la respuesta JSON sin depender de estado global.

## 2.3 Programacion Asincrona (Node.js)

Node.js usa un modelo de ejecucion de un solo hilo con un event loop. Las operaciones de I/O (lectura de archivos, ejecucion de inferencias) se manejan de forma no bloqueante mediante async/await y Promises, permitiendo atender multiples solicitudes concurrentes.

```
app.post("/query", async (req, res) => {  
    const query = normalizeQuery(req.body?.query);  
    // Carga asincrona del archivo .pl  
    const knowledgeBase = await loadKnowledgeBase();  
    // Inferencia asincrona con Tau Prolog  
    const solutions = await runPrologQuery(knowledgeBase, query);  
    return res.json(formatSuccess(solutions, query));  
});
```

Los callbacks internos de Tau Prolog se envuelven en Promises para poder usarlos con async/await, unificando el modelo asincrono del proyecto.

### 3 Arquitectura del Sistema

El sistema sigue una arquitectura de capas unica: un solo modulo Node.js que integra el servidor HTTP, la capa funcional de normalizacion y el motor logico.

| Capa          | Componente       | Tecnologia        | Paradigma   |
|---------------|------------------|-------------------|-------------|
| HTTP          | Express Router   | Node.js + Express | Asincrono   |
| Normalizacion | normalizeQuery() | JavaScript puro   | Funcional   |
| Inferencia    | runPrologQuery() | Tau Prolog        | Logico      |
| Conocimiento  | knowledge.pl     | Prolog            | Declarativo |
| Concurrencia  | async/await      | Promises          | Asincrono   |

#### Flujo de ejecucion

|   |                                                                                  |
|---|----------------------------------------------------------------------------------|
| 1 | Cliente envia POST /query con body: {"query": "penalty_applicable(X)."}.         |
| 2 | Express recibe la solicitud y llama al handler asincrono                         |
| 3 | normalizeQuery() (funcion pura) valida y formatea la entrada                     |
| 4 | loadKnowledgeBase() lee knowledge.pl de forma asincrona                          |
| 5 | runPrologQuery() crea sesion Tau Prolog, carga hechos y reglas, ejecuta consulta |
| 6 | El motor Prolog realiza inferencia por backtracking y retorna soluciones         |
| 7 | formatSuccess() (funcion pura) construye la respuesta JSON                       |
| 8 | El servidor retorna el JSON al cliente con status 200                            |

#### Estructura de archivos

```
motor-inferencia/
  server.js      <- Servidor HTTP + logica asincrona + funcional
  knowledge.pl   <- Base de conocimiento Prolog
  package.json  <- Dependencias del proyecto
  README.md     <- Instrucciones de instalacion y uso
  reporte.pdf   <- Este reporte
```

## 4 Ejemplo de Ejecucion

### 4.1 Inicio del servidor

```
$ node server.js

v Motor de Inferencia Logica corriendo en http://localhost:3000
  POST /query      -> ejecutar consulta Prolog
  GET  /examples   -> ver consultas de ejemplo
  GET  /health     -> estado del servidor
```

### 4.2 Consulta: penalizacion aplicable

```
POST /query
Content-Type: application/json

{ "query": "penalty_applicable(contract2)." }
```

```
{
  "success": true,
  "query": "penalty_applicable(contract2).",
  "results": [ { "result": "true", "bindings": null } ],
  "count": 1
}
```

El motor infiere que contract2 tiene penalizacion porque sus hechos indican 45 dias de mora (> 30), activando la primera clausula de penalty\_applicable/1.

### 4.3 Consulta con variable: contratos vencidos

```
{ "query": "overdue_contract(X)." }
```

```
{
  "success": true,
  "query": "overdue_contract(X).",
  "results": [ { "result": "true", "bindings": { "X": "contract2" } } ],
  "count": 1
}
```

#### 4.4 Tabla de consultas disponibles

| Consulta                  | Descripcion                    | Resultado esperado       |
|---------------------------|--------------------------------|--------------------------|
| penalty_applicable(X).    | Penalizacion aplicable         | X = contract2, contract4 |
| overdue_contract(X).      | Contratos vencidos             | X = contract2            |
| at_risk_contract(X).      | Contratos en riesgo (+10 dias) | X = contract2, contract4 |
| client_in_default(X).     | Clientes en incumplimiento     | X = client_b             |
| premium_client(X).        | Clientes premium               | X = client_a, client_c   |
| requires_intervention(X). | Casos criticos                 | X = client_b             |

## 5

## Conclusiones y Extensiones

---

### 5.1 Conclusiones

- **Complementariedad de paradigmas:** Los tres paradigmas no compiten; se complementan. Prolog expresa la logica de negocio en forma natural. Las funciones puras garantizan que el procesamiento sea predecible y testeable. `async/await` permite escalar sin bloquear.
- **Separacion de responsabilidades:** Cada capa tiene una responsabilidad clara. El archivo `.pl` contiene solo conocimiento. El servidor contiene solo orquestacion. Las funciones puras manejan solo transformacion de datos.
- **Tau Prolog como puente:** La biblioteca Tau Prolog permite integrar un motor Prolog completo dentro de Node.js sin procesos externos, simplificando el despliegue y la integracion asincrona.
- **Expresividad declarativa:** Agregar una nueva regla de negocio requiere unicamente modificar `knowledge.pl`, sin tocar el servidor. Esto demuestra la ventaja del enfoque declarativo para dominios cambiantes.

### 5.2 Extensiones posibles

- **Persistencia:** Reemplazar el archivo `.pl` por una base de datos que genere dinamicamente hechos Prolog al vuelo segun registros en PostgreSQL o MongoDB.
- **Autenticacion:** Agregar JWT middleware en Express para proteger el endpoint `/query` y limitar el acceso por usuario o rol.
- **Multiples bases de conocimiento:** Permitir que el cliente especifique en el request cual base de conocimiento usar (contratos, inventario, RRHH).
- **Interfaz grafica:** Desarrollar un frontend React que permita escribir consultas Prolog con autocompletado y visualizar el arbol de inferencia.
- **Testing:** Implementar pruebas unitarias con Jest para cada funcion pura y pruebas de integracion para los endpoints.
- **Despliegue:** Contenerizar la aplicacion con Docker y desplegarla en Railway, Render o AWS Lambda para uso en produccion.

---

**El proyecto demuestra que un sistema de inferencia logica puede integrarse de forma elegante en una arquitectura web moderna, aprovechando lo mejor de cada paradigma para construir software mas expresivo, modular y mantenible.**